UNIVERSIDAD DE SAN CARLOS DE GUATEMALA
FACULTAD DE INGENIERÍA
ESTRUCTURA DE DATOS
CATEDRÁTICO: ING. ESPINO
TUTOR ACADÉMICO: RUBÉN ALEJANDRO RALDA MEJIA



MANUAL TÉCNICO

OTTONIEL FABRICIO VÁSQUEZ PINEDA
CARNÉ: 202307820
SECCIÓN: B

GUATEMALA, 25 DE OCTUBRE DEL 2,025

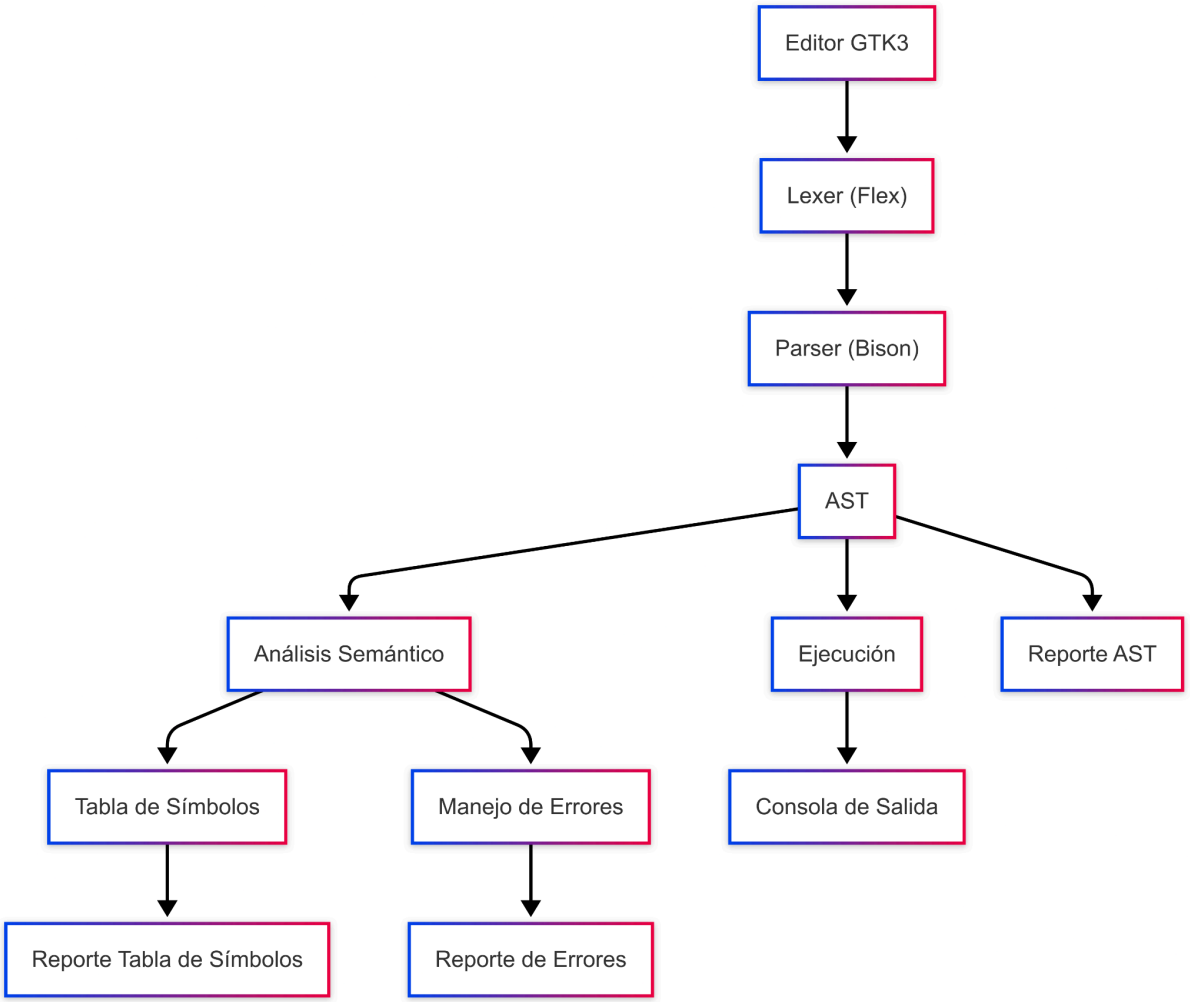
Introducción

El intérprete JavaLang es una herramienteducativa desarrollada en C con Flex y Bison que permite analizar, validar y ejecutar código escrito en un lenguaje académico similar a Java. Su objetivo es facilitar la comprensión de las fases de un compilador: análisis léxico, sintáctico, semántico, generación de AST y ejecución. Incluye una interfaz gráfica en GTK3 para mejorar la interacción con el usuario.

Arquitectura General

El sistema sigue una arquitectura modular pipeline:

1. Entrada: Archivo .usl o código desde el editor.
2. Lexer: Tokenización con Flex.
3. Parser: Construcción de AST con Bison.
4. Análisis Semántico: Validación de tipos y scopes.
5. Ejecución: Recorrido del AST.
6. Salida: Resultados en consola y reportes (AST, errores, tabla de símbolos).



Gramática formal de JavaLang

Lexer

%

System.out.println { yylval.str = strdup("System.out.println"); return TOKEN_PRINT;}

int

float

String

char

double

boolean

true

….

","

"."

Gramática Bison (parser)

program → main_class

main_class → "public" "static" "void" "main" "(" ")" block

block → "{" statement_list "}"

statement_list → statement_list statement | statement

statement → declaration ";"

| assignment ";"

| access_writing ";"

| print_statement ";"

| if_statement

| switch_statement

| while_statement

| for_statement
| vector_declaration ";"
| matrix_declaration ";"
| array_multi_declaration ";"
| void_function
| return_statement ";"
| function_with_params
| break_statement ";"
| continue_statement ";"

expression → expression "||" expression
| expression "&&" expression
| expression "|" expression
| expression "^" expression
…
| expression "%" expression
| "!" expression
| "-" expression
| "(" type ")" expression
| postfix

primary → NUMBER
| DOUBLE_LITERAL
| FLOAT_LITERAL
| STRING
| CHAR
| ID
| BOOLEAN
| "Integer.parseInt" "(" expression ")"

| "Double.parseDouble" "(" expression ")"

| "Float.parseFloat" "(" expression ")"

| "String.valueOf" "(" expression ")"

| "Arrays.indexOf" "(" expression "," expression ")"

| ID "." "length"

| ID "." "add" "(" expression ")"

| STRING "." "equals" "(" expression ")"

| ID "." "equals" "(" expression ")"

| "String.join" "(" expression "," expression_list ")"

| "String.join" "(" expression "," expression ")"

| access

| function_call

| "(" expression ")"

expression_list → expression_list "," expression | expression

Diseño de Módulos

Módulo Lexer (flex)

- Archivo: lexer.l
- Funcionalidad: Convertir el código fuente en tokens (unidades léxicas) mediante expresiones regulares.

Características

- Desarrollado con Flex
- Identifica palabras reservadas, operadores, literales, identificadores
- Maneja comentarios (// y /* */) y espacios en blanco
- Procesa secuencias de escape en strings y caracteres

- Coordina con el parser mediante tokens definidos

Funciones principales

- yylex(): Función principal que retorna tokens
- map_escape(): Convierte secuencias de escape a caracteres
- unescape(): Procesa strings con secuencias de escape

Módulo Parser

Archivo: parser.y

Funcionalidad: Analizar la estructura sintáctica según la gramática definida y construir el AST.

Características:

- Desarrollado con Bison
- Define la gramática completa de JavaLang
- Construye el AST durante el parsing
- Maneja precedencia de operadores
- Gestiona errores sintácticos con recuperación

Funciones:

- main_class: Punto de entrada (método main)
- declaracion, asignacion, expresion
- Estructuras de control: if, switch, while, for
- Funciones y llamadas a funciones
- Vectores, matrices y arrays multidimensionales

Módulo AST

Archivos: ast.h ast.c

Funcionalidad: Representar la estructura del programa en memoria para su posterior análisis y ejecución.

Características:

- Estructura de nodos con tipo, valor, hijos y ubicación
- Funciones para crear, liberar y manipular el AST
- Generación de gráficos con Graphviz
- Soporte para todos los tipos de nodos del lenguaje

Funciones principales:

- crearNodo(): Crea un nuevo nodo del AST
- agregarHijo(): Añade un nodo hijo
- liberarAST(): Libera memoria del AST
- graphvizAST(): Genera representación Graphviz
- exportarAST(): Exporta el AST a formato DOT/PNG

Módulo de Ejecución AST

Archivos: ejecutar_ast.h, ejecutar_ast.c

Responsabilidad: Recorrer e interpretar el AST para ejecutar el programa.

Características:

- Intérprete que ejecuta instrucciones representadas en el AST
- Manejo de declaraciones, asignaciones y expresiones
- Control de flujo (if, switch, while, for)
- Gestión de funciones y retornos
- Manipulación de arrays multidimensionales

Funciones principales:

- ejecutarNodoAST(): Ejecuta un nodo y sus hijos
- interpretarPrograma(): Función principal de ejecución

Módulo Manejo de Errores

Archivos: err.h, err.c, errorTIpos.h, errorTipos.c

Funcionalidad: Detectar, almacenar y reportar errores en todas las fases.

Características:

- Sistema unificado de errores sintácticos y semánticos
- Categorización de errores por tipo y gravedad
- Reportes detallados con ubicación (línea, columna)
- Gestión de memoria para registros de error

Funciones principales:

- addError(): Añade error semántico
- addErrorSintactico(): Añade error sintáctico
- mostrarErrores(): Muestra reportes de errores
- liberarErrores(): Libera memoria de errores

Módulo de Interfaz Gráfica

Archivo: Gtk.c , Gtk.h

- Construcción de la interfaz gráfica usando GTK
- Manejo de widgets (treeviews, textviews, botones)
- Funciones para llenar las tablas de símbolos y errores
- Manejadores de eventos (parsear, limpiar, cargar archivos)

Decisiones de Diseño

Manejo de memoria: El manejo de memoria lo útilice con malloc y free, en la cual este me ayuda a almacenar y también a liberar cache o variables o símbolos y entre otras cosas que ya no utilizare en memoría, esto me ayuda a tener una mejor ejecución en el programa.

Manejo de scopes: esto lo maneje en mi tabla de simbolos, en la cual lo utilice con un contador y con un string, para evaluar si está dentro de una función o ciclo y entre otras, y el contador para evaluar si cada cuanto scope está entrando.

Retos y Soluciones

Reto 1: Ambigüedad en la gramática

Solución: Reestructuración de reglas y trabajad de diferentes formas.

Reto 2: Print

Solución: Tener diferentes evaluaciones y que agarre las variables y absorba el valor con la tabla de símbolos.

Reto 3: Mucho código

Solución: Crear clases para varias funciones y tener una estructura mejor ordenada

Reto 4: Aprender cómo funciona el arm64

Solución: Con prueba y error y aprendiendo durante el proceso.

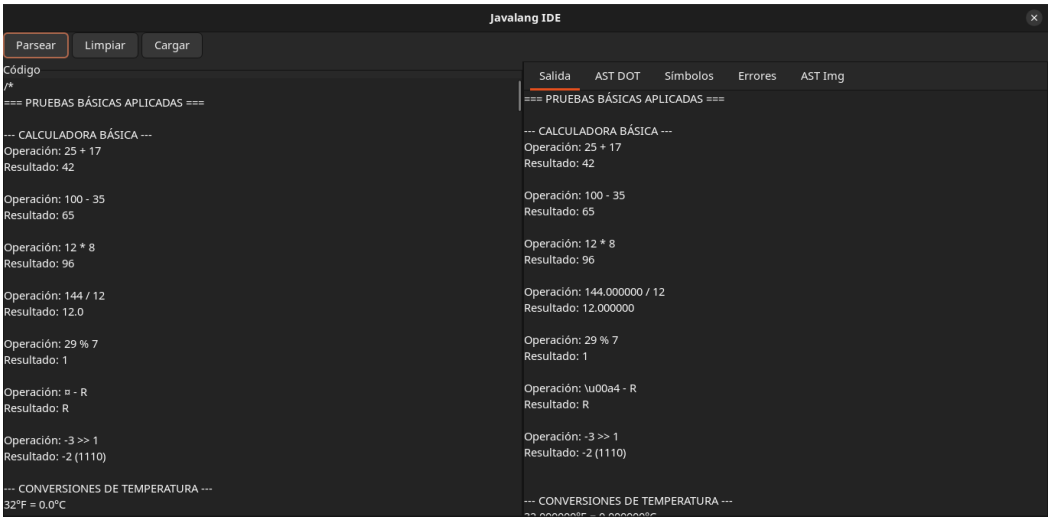
Pruebas y validación:

Pruebas unitarias: verificar que sirva el parser y el lexer

int y = 2;

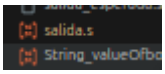
Variable ‘y’ ha sido declarado con valor 2.

Resultados y Métricas

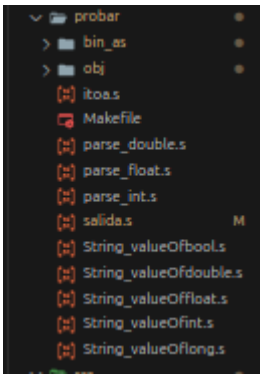


Generador de ARM64

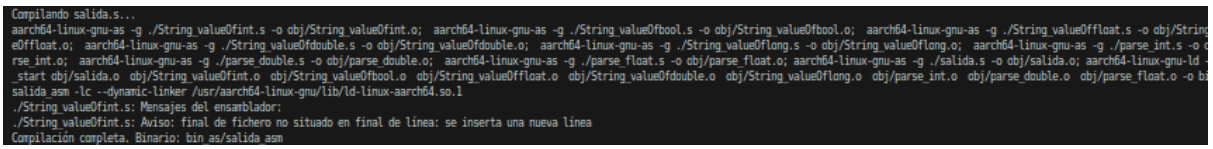
El Generador de ARM64 que se crea al darle parser al proyecto, este crea un archivo con su contenido de código arm64, pero en la cual si se puede compilar en qemu, el archivo se puede llamar salida.s



y este se puede compilar en qemu y tengo un apartado en la cual se puede ejecutar el código arm64, en la cual lo tiene que pasar a una carpeta llamada probar y ahí se pega el archivo o el contenido al archivo que ya está, y para ejecutarlo, tiene que abrir la terminal en la carpeta probar.



En la cual ya cuando lo abra, para ejecutarlo tiene que poner make archivo=salida.s y éste mostrará mensajes de éxito o un mensaje de error por si tiene error el código de arm64.



Después le da al comando de make run para poder ejecutar el código de arm64 si no hay ningún error, si hay error no se podrá ejecutar.

